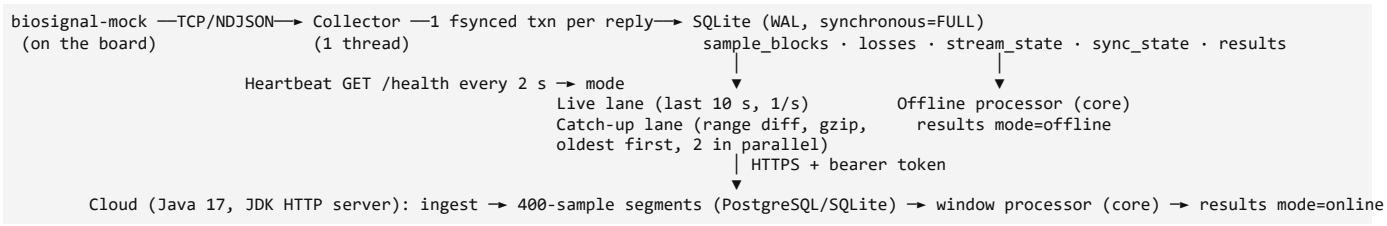
Design note

Validated against stand-in versions of `biosignal-mock` and `check_ground_truth` written from the assessment's "Mock service" section; the official tools were not available. **A real Raspberry Pi 4 run was not possible:** ARM64 figures come from a GitHub Actions ubuntu-24.04-arm native ARM64 runner, limited to 4 CPUs / 2 GB; not a Raspberry Pi 4; power cuts simulated by SIGKILL.

Architecture



Device. The collector polls each sensor every 0.5 s (back to back while behind), always asking `from_index = next expected index`. A reply is accepted only if the CRC-32 of the raw `data` text as received (never re-serialised) matches, rows have the right width and values are 24-bit. Each accepted reply becomes **one SQLite transaction**: one compact row with the new samples (delta/varint), the loss records for any gap before them, and the sensor's `next_index`. Duplicated or late replies are self-describing (`first_index`), so they can never shift data; a block may only start at or after the committed `next_index` and any overlap is trimmed inside the transaction, so a sample is stored at most once. Stalls hit a 2 s monotonic timeout and are retried; a sensor that stops answering while the other answers is polled less often so it cannot starve the healthy one; after repeated error replies the client re-checks the session. A new `session_id` closes the old session with `[next_index, null]` declared lost. The collector never touches the network.

Loss ledger. For every session and sensor, every index below `next_index` is stored or inside exactly one loss record with a reason (the mock's `lost` reason where it gave one, e.g. `buffer_overflow`, `sensor_outage`; otherwise `not_delivered_by_mock`, `mock_session_restarted_unread_tail` or `disk_full`). Nothing is filled in. `pipeline verify` checks this invariant and SQLite's integrity.

Cloud. Batches merge idempotently into 400-sample segments (presence bitmap + compressed values; a duplicate never overwrites). A window is computed only once all its inputs are *resolved* (held, declared lost, or past the end of a closed session), so every stored result is final. Device and cloud run the same `core` code; heart-rate peaks equal a scan from index 0 (local anchor search, cross-checked against a global scan in tests).

Switching and catch-up. Two failed heartbeats → offline, two good ones → online; in-flight uploads are aborted on every switch, and failed DNS lookups are not cached (the JVM default would hold a failure for 10 s after the network returns). Offline, the device computes windows itself. Back online, the live lane restarts at "now − 10 s" (so live windows have their 5 s heart-rate context) while the catch-up lane asks which ranges the cloud holds and sends the difference in 40,000-index gzip batches, halving the batch on timeout and backing off on errors.

Interpretations (each switchable by one config key, same value on device and cloud): rounding is round-half-even on the exact double (Python `round`); only sensor 1 gaps null the heart rate; result fields are flat (`mean_1`, `min_1`, …); a window with no data on a sensor is reported with `missing_N = 400` and nulls; the last window of a closed session is reported with indices past the end counted as missing; windows restart at 0 in each session.

Measured (stand-ins, local network): online results 0.51 s (median) and at most 1.5 s after a window ends; offline 5.5 s after a network cut, online 1.0 s after it returns; a 60 s outage fully synced 2.0 s later; a 2-hour backlog (5.77 M samples, 28.5 MB on the wire) synced 11 s after restore, and **156 s with the cloud limited to 0.1 CPU / 512 MB** on PostgreSQL; three `kill -9` restarts, every injected mock fault, disk full and ±1 h clock jumps with zero silent loss, zero duplicates and every result equal to an independent reference. **Live cloud** (GitHub Actions ARM64 runner to Render's free tier and Neon over the internet, runs 1 and 2 before and after the polling fix below, run 3 a soak only; every data-integrity check passed in all three): online results p50 772 / 988 / 1130 ms, p95 1490 / 1264 / 1441 ms over 3,570-3,571 windows an hour; max 2116 ms in run 1 and 4074 ms in run 3, but **one window in run 2 took 5.5 s** (below); offline 3.5-3.6 s after a network cut, online 3.8-3.9 s after it returns; the 2-hour backlog (5.76 M and 5.77 M samples) caught up 84-85 s after restore. Both backlog tests are **accelerated**: the outage's data is generated at about 20x speed and the stand-in's buffer raised to 200 s of data, so 2 hours of data build up in about 6 minutes. Render's wake-up took **22.9-24.1 s** (five measurements), so after an outage longer than 15 minutes the switch back to online can take about 25 s unless an uptime pinger keeps the service awake.

Online latency near-miss (run 2). Window 3088 (ending about 21:47 IST) reached the cloud after 5,497 ms, and the device's upload backlog peaked at 3,601 samples (about 4.5 s of data; 1,209 in run 1). That fits one ingest round trip taking about 4.5 s. It was not a timeout and retry: the live upload timeout is 20 s and the complete device log shows no failed upload and no mode switch. The device did not stall (no commit gap over 3.5 s, which was the mock's session restart), and the cloud logged no error, did not restart and its database stayed up. Render's free tier keeps no per-request timings and Neon recorded no query statistics, so whether the network, Render or Neon held the request is **not determined**, and no pipeline behaviour was changed for it. **Run 3** repeated only the soak, on a commit that adds logging only: 0 of 3,570 windows over 5 s (max 4,074 ms, p99 1,564 ms). Six live uploads took 2.0-4.3 s, each in one attempt with the whole wait before the first response byte, and every window over 2 s but one traces to them. In two the cloud finished storing the batch 2.2 s and 3.8 s after it was sent (normally 0.14 s); in three it stored it within 0.18 s and the reply took 1.8-4.1 s more. Slow round trips therefore occur on either leg; where inside a leg (network, Render's proxy, the cloud process or Neon) is still not recorded. Over the three runs the 5 s target was met by 10,711 of 10,712 windows on this host, not guaranteed.

What is guaranteed, and under which assumptions

Claim	Basis	Assumes
A sample acknowledged by a commit survives a crash, reboot or power cut	WAL with `synchronous=FULL` fsyncs every commit	the storage honours flush (some SD cards do not)
Every sample is on disk at most once	blocks start at or after the committed `next_index`; overlap trimmed in the same transaction	the database is not corrupted
No silent loss: every missing index is listed with a reason	ledger invariant, written in the same transaction	as above
A daemon restart loses nothing if it is back within ~8 s	resume from `next_index` via `from_index`; the mock buffers 10 s	the mock kept running (systemd restarts in ~1 s)
Offline within 10 s, online within 10 s of the network changing	≤ 2 s + 2 s + 1.5 s to go offline; ≤ ~4 s to go online	the cloud is awake (Render's free tier sleeps after 15 idle minutes and took 23-24 s to wake)
Cloud copy complete after an outage	range reconciliation repeats until nothing is missing	the cloud is reachable and fast enough
Results identical online and offline	same code, computed only on resolved inputs	the interpretations above match the reference

Not guaranteed. (1) Data the mock generated but we had not yet committed when power is lost. (2) The unread tail of an interrupted session is listed only once the device sees the new session. (3) If the disk stays full beyond a 2-minute memory buffer, the oldest buffered samples are dropped and declared `disk_full`; if the process dies while buffering, those samples later appear as the mock's `buffer_overflow`. (4) Online latency and the switch back to online

depend on the cloud host; on Render's free tier the service sleeps after 15 min without requests (an uptime pinger prevents it). (5) One cloud instance only. (6) Wall-clock time is display-only; all timeouts and ordering use the monotonic clock. (7) **Reboot and start at boot were not tested**: `install.sh` was run only in a container with a fake `systemctl`, so the systemd units never ran; restarts were tested by kill -9 and simulated power cuts only.

Maximum data loss on a power cut

Data reaches disk only by commit, and a commit happens every poll (0.5 s when caught up) plus the time to read and commit one reply (normally 5-30 ms). At the moment of a power cut the samples generated since the last commit are either in the mock's memory or in ours, and both are gone (the mock runs on the same board and restarts with a new session). The loss is therefore **one poll interval plus one read-and-commit, about 0.5-0.6 s per sensor**. **Power-cut results** (`powercut`, SIGKILL of daemon and mock together, never a real power pull): worst loss 0.20 s per sensor locally, 0.44 s and 0.19 s in ARM64 runs 1 and 2 (run 3 ran only the soak). **Not power-cut results**, but the longest time without a commit in other scenarios (a power cut then would lose the data generated in it): 0.50-0.52 s in most; under load 0.54 s (local backlog, cloud at 0.1 CPU), 0.55 s (run 2 backlog), 0.61 s (run 2 kill -9) and 1.0 s (run 1 backlog, data generated at 20x); 0.77-1.15 s in the clock tests, where libfaketime slows every JVM about 2x; 3.5 s at each soak's mock restart, mostly while no data was being generated (the old session had ended). Injected disk-full and mock stalls stop commits by design (16.5-21.8 s; see "Not guaranteed" (3)). We state **at most 1.0 s per sensor in steady operation**, about 0.5 s above what was measured there; under load the gap reached the 1.0 s bound itself, and a slower Pi was not measured. The lost range is recorded at the next boot as `[next_index, null]`, reason `mock_session_restarted_unread_tail`. The bound assumes the storage honours fsync and the disk was not full.

Resource use (online, mock with `--duration 3600`)

Environment (none is a Pi 4)	Avg CPU (of 4 cores)	Peak RSS	Disk written	Network sent
amd64 container, native x86-64 laptop, 4 CPUs / 2 GB, 15 min, local cloud	0.6 % (2.3 % of one core)	109 MB	176 MB/h	15.6 MB/h
ARM64 container under QEMU emulation on that laptop, 4 CPUs / 2 GB, 1 hour	1.6 % (6.3 % of one core)	270 MB	173 MB/h	15.2 MB/h
GitHub Actions native ARM64 runner (Neoverse-N2), 4 CPUs / 2 GB, 1 hour, live cloud, run 1, before the polling fix	0.27 % (1.07 % of one core)	120.5 MB	264.55 MB/h	16.05 MB/h
Same runner type, run 2, after the polling fix	0.25 % (1.00 % of one core)	124.2 MB	178.69 MB/h	16.30 MB/h
Same runner type, run 3, soak only, logging-only commit	0.24 % (0.95 % of one core)	124.8 MB	183.30 MB/h	16.05 MB/h

How they relate to a Pi 4 (Cortex-A72, 1.5-1.8 GHz, SD card): the amd64 and native-ARM64 CPU figures **understate** Pi 4 load, because both run on cores several times faster than a Cortex-A72; the emulated CPU and RSS are **inflated** by binary translation relative to native execution on the same laptop, and the Pi's slower cores push the other way, so that row bounds nothing. RSS, bytes written and bytes sent depend on the code and settings, not the core, so the native-ARM64 row (same instruction set and JVM as the Pi) is the best estimate of those; an SD card's internal write amplification is not visible in any row.

Measured with `scripts/measure-resources.sh` (`/proc/PID/stat`, sampled RSS, `/proc/PID/io` `write_bytes`, `/proc/net/dev`; elapsed time from `/proc/uptime`). Writes are SQLite's: one fsynced transaction per reply (~4/s), each rewriting a few 1 KB pages in the WAL and again at checkpoint. Storing one compact row per reply instead of one row per sample cut writes from 336 to 176 MB/h at the same CPU; 1 KB pages had already cut them from 573 (these runs: `evidence/write-rate-history/`). **Why 265 vs 176 MB/h:** not the filesystem, journaling or SQLite settings (all rows: ext4 under overlayfs, same pragmas; `write_bytes` excludes the ext4 journal). The CI start order on the same laptop kernel that gave 176 MB/h gave 286 MB/h. The collector re-reads at once after a reply at least 90 % the size of the largest seen, to drain a capped backlog; started right after the mock, the daemon never saw a full 1 s reply, so every 0.5 s reply qualified and replies and fsynced transactions doubled (8/s, not 4). Fixed after the run: a re-read is skipped when one after an equally large reply just returned almost nothing (re-checked every 10 s). Every reply is still committed, so durability and the power-cut bound are unchanged. Same laptop, same start order: **186 MB/h**; native ARM64: 264.55 MB/h before, **178.69 MB/h** after and 183.30 MB/h in run 3 (`.github/workflows/arm64-evidence.yml`, runs 38730796747, 38759716475 and 38788955634; `evidence/SUMMARY.md`).